

Relatório 8 - Prática: Web Scraping com Python p/ Ciência de Dados (II)

Gustavo Nunes da Silva

1. Introdução

Nesse card eu quis aprender web scraping com Python, que é basicamente pegar o HTML de uma página e pegar dali só a informação que interessa. Eu assisti ao vídeo do Keith Galli usando BeautifulSoup e requests, e depois fiz a mesma coisa em outra página pra ver se eu tinha entendido mesmo.

2. Desenvolvimento

O vídeo começa mostrando o HTML da página antes de abrir o Python, pelo botão direito em "exibir código fonte". Eu achei importante porque toda a aula depois disso é basicamente aprender a navegar dentro dessa estrutura de tags. Pra carregar a página no Python eu uso o `requests.get(url)`, que traz o HTML bruto, e depois o `bs(r.content)` transforma isso num objeto soup, que é o que permite navegar e buscar dentro do HTML como se fosse uma árvore.

As primeiras formas que eu usei pra buscar foram o `find` e o `find_all`. O `find` pega só a primeira vez que a tag aparece, e o `find_all` pega todas e coloca em uma lista. Os dois aceitam filtro por atributo, tipo `id` ou `classe`, e dá pra fazer busca em cascata, isolando primeiro o `body`, depois uma `div` dentro dele, depois um `h1` dentro da `div`, até chegar no que eu quero. Também dá pra usar `regex` com `re.compile` pra buscar um pedaço do texto sem saber a palavra exata, e isso me ajuda bastante quando a página tem muita coisa parecida.

Depois vem o `select`, que funciona igual seletor de CSS. A sintaxe muda bastante: "`div p`" pega parágrafo dentro de `div`, "`h2 ~ p`" pega parágrafo que vem logo depois de um `h2`, e "`body > p`" pega só o filho direto do `body`. Essa diferença entre filho direto e qualquer coisa mais abaixo foi uma das partes que mais me confundiu, porque o símbolo muda o resultado inteiro. No meu notebook da aula apareceu um exemplo que mostra isso eu escrevi `soup.select("[align=middle]")` pra pegar um elemento pela propriedade, mas o atributo certo do HTML é `align`, não `algn`. O código roda sem erro nenhum, só devolve uma lista vazia, porque o BeautifulSoup não acha nada com esse nome e não avisa que o seletor está errado (Figura 3).

Pra tirar o valor de dentro da tag tem duas formas `.string`, que só funciona se a tag não tiver outra tag dentro, e `get_text()`, que pega o texto de todos os filhos juntos. Pra pegar um atributo, tipo o link de um "`a`", trata a tag como se fosse um dicionário, com `link["href"]`. Isso é bem diferente do texto, e a mistura das duas coisas foi outro ponto que me confundiu. A navegação por `parent`, `child` e `sibling` completa essa parte, servindo pra

quando a página não tem id único e a única forma de achar um dado é a partir de um vizinho dele.

Na parte de exercícios, o vídeo passa a raspar os links de rede social de três formas diferentes, todas chegando no mesmo resultado uma usando select direto, outra achando a lista com find e depois os links dentro dela, e a terceira selecionando direto pelo item da lista. Isso mostrou que quase sempre existe mais de um caminho certo pra chegar no mesmo dado, e escolher um ou outro é questão de gosto e de quão detalhado o seletor precisa ser.

O exercício da tabela foi a parte que mais juntou tudo. Primeiro pega os nomes das colunas dentro do thead, depois percorre cada linha do tbody puxando as células com find_all, e por fim monta um DataFrame do pandas com essa lista. Com isso eu consegui ver pra que o scraping serve na prática o dado sai bagunçado da internet e vira uma tabela organizada, pronta pra ser filtrada ou analisada. O exercício das curiosidades usando a palavra "is" tem exatamente o mesmo tipo de erro que achei no seletor do align eu escrevi select("ul.fun-facts is") quando o certo seria "ul.fun-facts li", já que os itens da lista de curiosidades estão em tags li e não numa tag chamada is. E de novo eu tive um resultado vazio sem nenhum aviso de erro, o que mostra de novo que seletor CSS errado é um dos jeitos mais silenciosos de um scraping falhar (Figura 4).

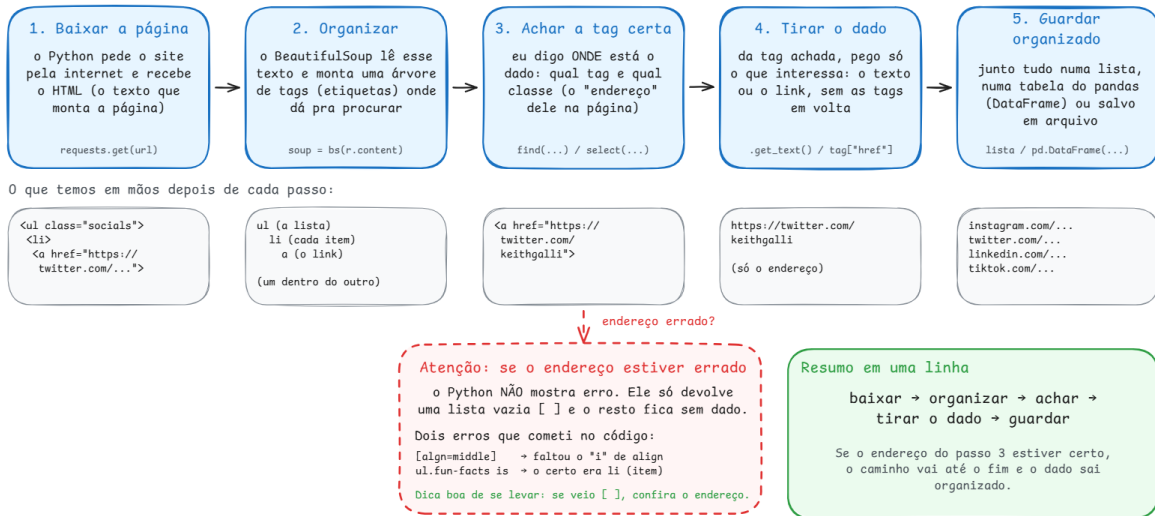
No final o vídeo baixa uma imagem montando a URL completa a partir do atributo src e gravando o conteúdo em bytes num arquivo local, e depois resolve um desafio de percorrer dez links em loop, tirando uma palavra de cada página, até formar uma frase inteira juntando tudo. Esse desafio final foi o que mais pareceu com scraping de verdade pra mim, porque não é mais sobre uma página só, é sobre repetir o mesmo processo em várias páginas de forma automática.

Pra praticar usei o books.toscrape.com, que é um site feito de propósito pra treinar scraping. Eu usei o find e o select de novo pra pegar o primeiro livro da lista e os gêneros da barra lateral, usei regex pra contar quantos títulos têm a palavra "the", e peguei a mensagem de aviso do site com get_text. A parte que eu fiz além foi pegar a tabela de informação do produto de um livro específico e transformar em DataFrame, e baixar imagens, que são exatamente os dois exercícios que dão mais trabalho no vídeo. No lugar da capa do livro eu baixei as fotos de uma Ferrari 812 e de uma Lamborghini SVJ da página da Super Veloce na Wikipédia, achando cada foto pela legenda dela (Figura 5). Nessa parte eu precisei mandar um User-Agent junto com o requests, porque a Wikipédia pode bloquear quem faz a requisição sem se identificar. No final eu fiz um loop simples pegando o preço de todos os livros da primeira página, parecido com o desafio da mensagem secreta, só que sem precisar entrar em cada link.

Figura 1

O caminho do web scraping: do site até o dado organizado

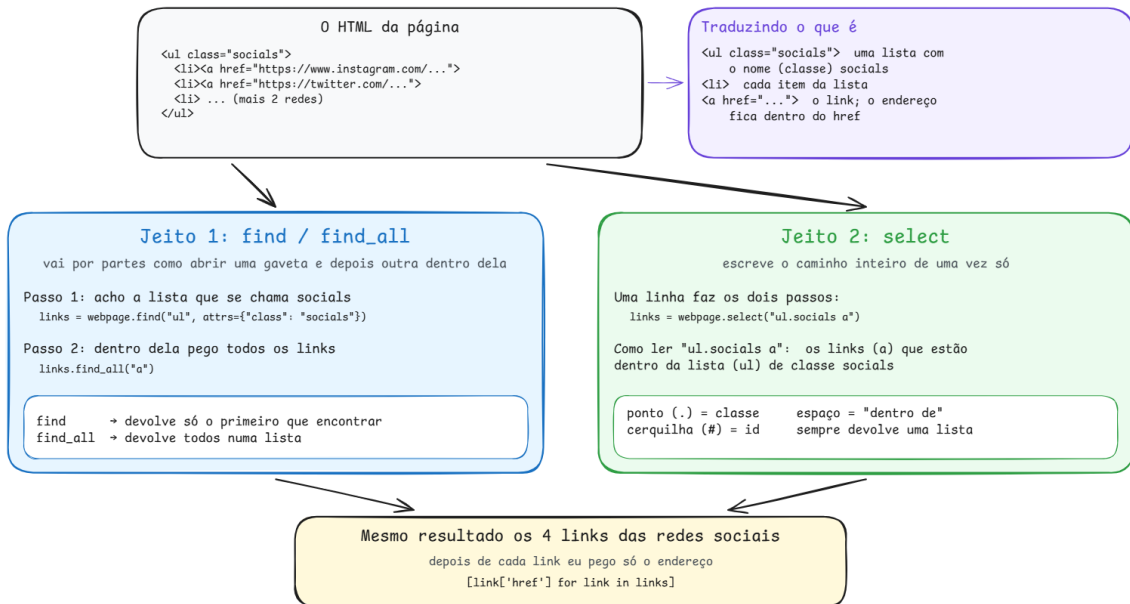
São 5 passos



Fonte – Autoria própria (elaborado no Excalidraw).

Figura 2

Dois jeitos de achar a mesma coisa find x select



Fonte – Autoria própria (elaborado no Excalidraw).

Figura 3

```
In [13]: #aqui eu digitei algn em vez de align e o certo seria [align=middle]
# o código roda normalmente sem erro nenhum, so devolve lista vazia porque não acho nenhum atributo "algn"
soup.select("[align=middle]")
```

Out[13]: []

Fonte – Autoria própria (print do Jupyter Notebook).

Figura 4

```
In [23]: import re

# aqui eu digitei is em vez de li no seletor e o certo era "ul.fun-facts li"
# como não existe tag <is>, o select devolve lista vazia e o resto do código não acha nada
facts = webpage.select("ul.fun-facts is")
facts_with_is = [facts.find_all(string=re.compile("is")) for fact in facts]
facts_with_is = [fact.find_parent().get_text() for fact in facts_with_is if fact]
print(facts_with_is)
```

[]

Fonte – Autoria própria (print do Jupyter Notebook).

Figura 5

```
In [8]: # troquei a capa do livro por carro: pego a foto da 812 e da svj na wikipedia em portugues
from IPython.display import Image, display

# a wikipedia pede um user-agent dizendo quem ta fazendo a requisicao, senao pode bloquear
headers = {"User-Agent": "card8-bootcamp-scraping/1.0 (projeto de estudo)"}

# as duas fotos estao na mesma pagina, entao baixo a html uma vez so
r = requests.get("https://pt.wikipedia.org/wiki/Super_Velocidade", headers=headers)
pagina_carros = bs(r.content)

# pra cada carro: nome do arquivo e um pedaco da legenda que identifica a foto
carros = {
    "Ferrari_812.jpg": "Ferrari 812",
    "Lamborghini_svj.jpg": "Aventador SVJ",
}

for arquivo, legenda in carros.items():
    # acho a figura cuja legenda tem o nome do carro e pego a imagem de dentro dela
    imagem = pagina_carros.select_one(f"figure:has(figcaption:~soup.contains('{legenda}')) img")
    caminho_imagem = imagem["src"] # vem sem o https, tipo //upload.wikimedia.org/...

    url_completa = "https:" + caminho_imagem.split("?")[0] # completo o endereço e tiro os parametros da fim
    url_completa = re.sub(r"/\d+px-", "/1280px-", url_completa) # peço uma versão maior da foto

    img_data = requests.get(url_completa, headers=headers).content
    with open(arquivo, "wb") as handler:
        handler.write(img_data) # gravo a imagem na pasta local

    print(f"[arquivo]: {len(img_data)} bytes")
    display(Image(filename=arquivo, width=500)) # mostro a foto aqui mesmo no notebook
```

ferrari_812.jpg: 411151 bytes



lamborghini_svj.jpg: 382199 bytes



Fonte – Autoria própria (print do Jupyter Notebook).

3. Conclusão

O que mais ficou pra mim nesse card foi que scraping não é sobre decorar método, é sobre entender a estrutura da página antes de escrever qualquer linha de código. O que mais me chamou atenção foram os dois seletores errados que apareceram no meu notebook da aula, porque nenhum dos dois deu erro, os dois só devolveram lista vazia. Isso mostrou que scraping falha de forma silenciosa, sem exceção pra avisar, e que testar o resultado de cada célula é tão importante quanto escrever o seletor certo. No final eu percebi que o processo é sempre parecido pegar o HTML, filtrar até chegar no que eu quero, tirar o texto ou o link, e organizar isso numa estrutura que dá pra usar depois, como um DataFrame ou um arquivo salvo.

4. Referências

GALLI, Keith. Comprehensive Python Beautiful Soup Web Scraping Tutorial! (find/find_all, css select, scrape table). YouTube, 11 jul. 2020. Disponível em: https://www.youtube.com/watch?v=GjKQ6V_ViQE.

Books to Scrape - Sandbox. Disponível em: <https://books.toscrape.com/>. Acesso em: 20 set. 2026.

SUPER VELOCE. In: WIKIPÉDIA, a enciclopédia livre. Disponível em: https://pt.wikipedia.org/wiki/Super_Veloce.